Lecture 9: Kernel I — Processes, System Calls & Exceptions

Session 11 · Mon Oct 5, 2026

Contents

The Instruction That Leaves Your Program . . . 2
Where We Are . . . 2
Run It: A Forbidden Instruction . . . 2
What Happened, and What a Kernel Panic Is . . . 2
User Mode and Kernel Mode . . . 3
Three Ways Into the Kernel . . . 3
An Exception Is a Jump the Hardware Takes . . . 4
What an Exception Handler Is . . . 4
Walk Through: What the CPU Does on an Exception . . . 4
Four Classes of Exceptions . . . 4
Interrupts: A Device Needs Attention . . . 5
Faults: The Instruction Gets Another Try . . . 5
Run It: Counting Page Faults . . . 5
Run It: Dividing by Zero on Two CPUs . . . 5
Traps: System Calls Are Numbered . . . 6
Arguments Go in Registers . . . 6
Hello, World With No C Library . . . 6
Run It: Two System Calls, Nothing Else . . . 7
What `syscall` Does, Step by Step . . . 7
How a System Call Reports an Error . . . 7
Run It: 1000 `printf` Calls, How Many `write`s? . . . 8
Run It: What a System Call Costs . . . 8
A Process Is a Running Program . . . 8
Run It: Every Process Has a Number . . . 8
One CPU, Many Processes . . . 9
The Context Switch . . . 9
A `read` That Has to Wait . . . 10
Run It: Counting Context Switches . . . 10
What the Context Switch Counts Show . . . 10
AI Systems Connection . . . 10
Think About It: `printf` on the GPU . . . 11
`printf` on the GPU: The CPU Does the Printing . . . 11
The Lecture in One Slide . . . 11
Practice . . . 12
Practice II . . . 12
Answers to Practice . . . 12
Answers to Practice II . . . 12
Reading . . . 12

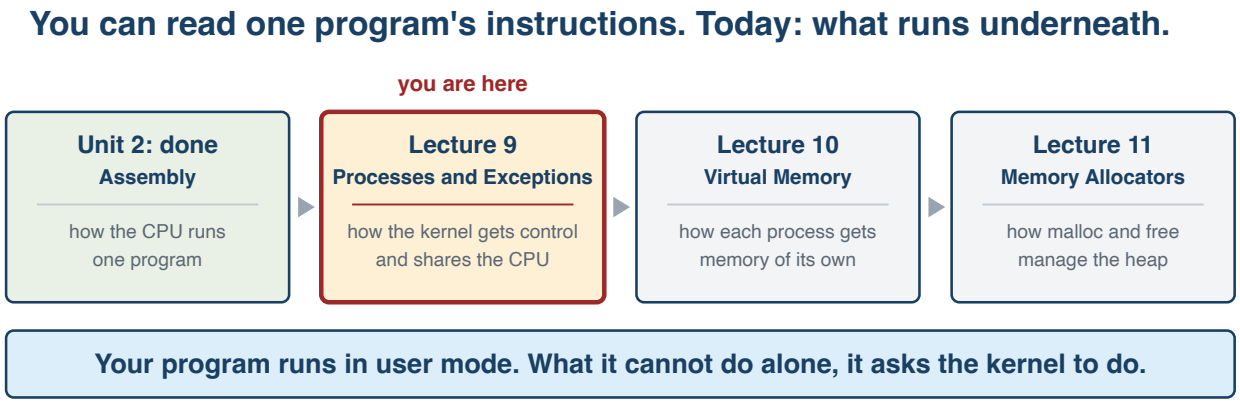
The Instruction That Leaves Your Program

This is the C library's `write` function, disassembled on Linux x86-64. Every `printf` you have written ends up here:

```
__write:
    movl  $1, %eax          # put the number 1 in %rax
    syscall                 # hand control to the operating system
    cmpq  $-4096, %rax      # did the operating system report an error?
    ja    error_path        # if so, go and set errno
    ret                     # if not, return the result
```

At `syscall` your program stops running its own code, and the operating system's kernel runs instead. Today: how control gets into the kernel, what the kernel does there, and how it shares one CPU among many programs.

Where We Are



Unit 2 treated your program as if it had the machine to itself. From now on it shares the CPU and memory with every other program, and the kernel is the software that makes the sharing work.

Run It: A Forbidden Instruction

A user program cannot command a device, read another program's memory, or stop the timer. The CPU itself enforces this. `hlt`, which halts the CPU, is reserved for the kernel. This program, `01-privileged.c`, reads its privilege level, then runs `hlt`:

```
unsigned short cs;
__asm__("movw %%cs, %0" : "=r"(cs));     // copy the %cs register into cs
printf("privilege level: %d  (3 = user mode, 0 = kernel mode)\n", cs & 3);
fflush(stdout);
printf("running hlt...\n");
fflush(stdout);                          // get both lines out before we die
__asm__ volatile("hlt");                 // only kernel mode may run this
printf("still running\n");              // never printed
```

```
$ make linux && ./01-privileged      # Linux x86-64 only
privilege level: 3  (3 = user mode, 0 = kernel mode)
running hlt...
Segmentation fault (core dumped)
```

What Happened, and What a Kernel Panic Is

The privilege level is the low two bits of the `%cs` register, and it was 3. At `hlt` the CPU refused, raised a *general protection fault*, and ran the kernel's handler. The handler ended one process. Every other program on the

machine kept running.

Side knowledge: the kernel panic. Now suppose the faulting instruction is in the kernel itself, for example a device driver that uses a bad pointer in kernel mode. The CPU raises the same kind of fault, but there is nothing above the kernel that can end the kernel. So the kernel stops the whole machine and prints what it knows. Linux calls this a **kernel panic**. Windows shows the blue screen. macOS shows a gray screen that says you need to restart.

A fault in user mode costs one process. A fault in kernel mode can cost the whole machine. That is one reason kernel code is kept small and reviewed carefully.

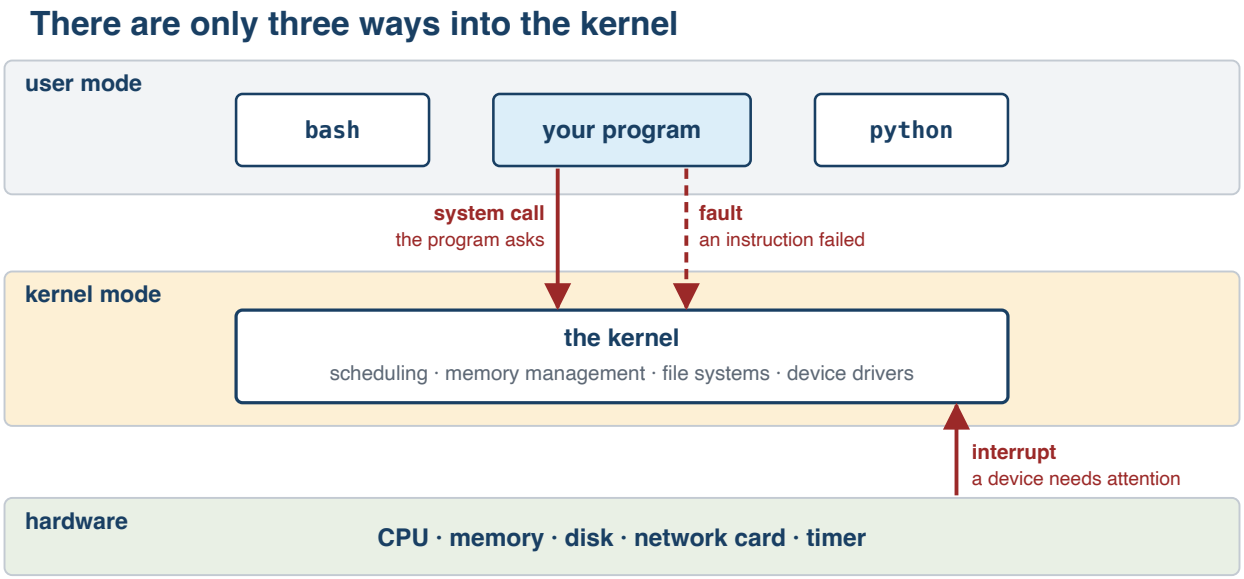
User Mode and Kernel Mode

The CPU is always in one of two modes:

- **User mode**, where your programs run. Privileged instructions and the kernel's memory are off limits.
- **Kernel mode**, where the **kernel**, the core of the operating system, runs. Everything is allowed.

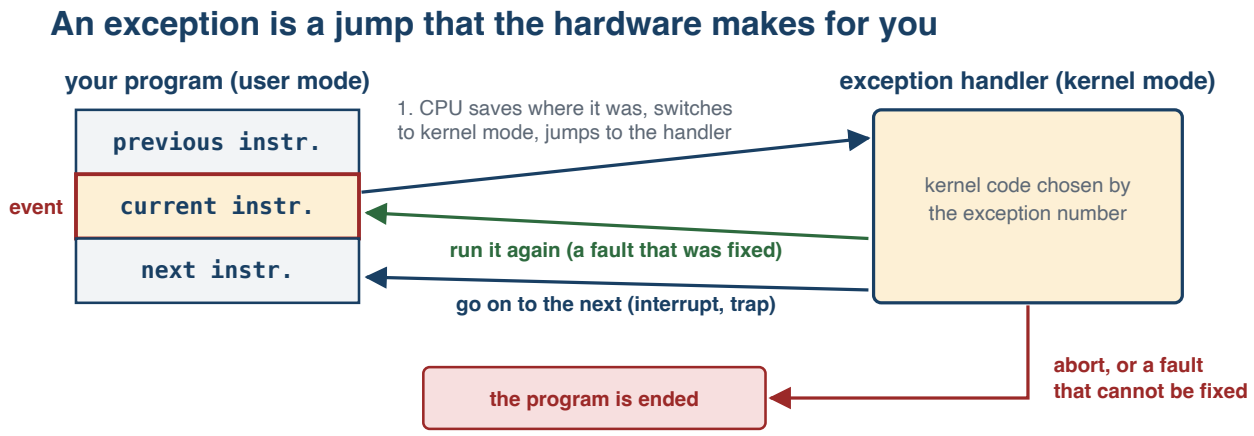
The kernel is not a separate program running next to yours. It is code that runs only when something switches the CPU into kernel mode. A program cannot do that by writing to a register. The only way in is an **exception**, which always jumps to code the kernel chose in advance.

Three Ways Into the Kernel



Control enters the kernel in one of three ways. The program asks the kernel for a service, which is a system call. One of the program's instructions fails, which is a fault. Or a device needs the CPU's attention, which is an interrupt.

An Exception Is a Jump the Hardware Takes



An **exception** is a transfer of control from the program to the kernel, made by the CPU when some event happens. The kernel code it runs is the **exception handler**.

What an Exception Handler Is

Why they are needed. The CPU can detect an event, but it cannot decide what to do about it. Only software knows whether a faulting address belongs to the program, which process should run next, or what a system call number means. The handler is where that decision is written down.

What they are. Ordinary functions inside the kernel, written in C with a little assembly. At boot the kernel writes their addresses into the exception table. On Linux x86-64 every page fault lands in `exc_page_fault`, and every `syscall` lands in `do_syscall_64`.

What is special about them. The CPU starts them, not a `call` instruction, so the program never names a handler and cannot supply its own. They run in kernel mode on the kernel's stack. They must save and restore every register they touch, because the interrupted program was not expecting a call. They return with `iretq`, which also restores the flags and the mode.

Walk Through: What the CPU Does on an Exception

The CPU does these steps itself. No instruction in your program asks for them.

1. It works out the **exception number**, from 0 to 255, for this event.
2. It saves the return address, flags and stack pointer on the kernel's stack.
3. It switches to kernel mode.
4. It jumps to the handler listed for that number in the **exception table**, which the kernel filled in at boot.

When the handler finishes, `iretq` restores the saved state and user mode. It is like `call` and `ret`, except the return address can be the *current* instruction, so a failed one can run again.

Four Classes of Exceptions

Way into the kernel	Exception class	Where it comes from	Where control goes afterwards
system call	**trap**	the program, on purpose (`syscall`)	the next instruction

Way into the kernel	Exception class	Where it comes from	Where control goes afterwards
fault	**fault**	the program, by accident (an instruction fails)	the same instruction runs again if the kernel fixed the problem; otherwise the program is ended
interrupt	**interrupt**	a device outside the program	the next instruction
hardware error	**abort**	broken hardware that cannot be fixed	nowhere; the program is ended

Aborts are rare, so the next slides skip them.

Interrupts: A Device Needs Attention

A disk finishes a read, a packet arrives, or the timer chip counts down. The device signals the CPU, which finishes its current instruction, runs the kernel's handler, and returns to the next instruction.

The **timer interrupt** matters most. It fires every few milliseconds, whatever is running. That is how the kernel gets the CPU back from a program stuck in `while (1) { }`.

Faults: The Instruction Gets Another Try

A **fault** happens when an instruction cannot finish. The handler either fixes the problem and runs the *same* instruction again, or ends the program.

The main example is the **page fault**, exception 14. The first time a program touches memory the kernel has not set up yet, the instruction faults, the kernel sets up the memory, and the instruction runs again. This is normal. If the address is one the program may not use at all, the kernel ends it: a *segmentation fault*.

Run It: Counting Page Faults

This program, `02-page-faults.c`, gets 64 MiB from `malloc`, writes one byte into each page, and asks the kernel how many page faults it handled:

```c
long size = 64 * MB;
long page = sysconf(_SC_PAGESIZE);     // 4096 on Linux x86-64
volatile char *p = malloc(size);       // volatile: every write must happen

long before = minor_faults();          // getrusage(...), then u.ru_minflt
for (long i = 0; i < size; i += page)
    p[i] = 1;                          // one write per page: the first touch
long after = minor_faults();
```

```
$ ./02-page-faults                  # Linux
page size          4096 bytes
pages touched      16384
minor page faults  16383
```

`malloc` handed back addresses, but the kernel supplied real memory only at the first write to each page. Each of those writes faulted and ran again, and the program never noticed.

Run It: Dividing by Zero on Two CPUs

```c
volatile int zero = 0;      // 03-divide.c; volatile: really divide at run time
int x = 42;
printf("about to compute 42 / 0\n");
fflush(stdout);
int q = x / zero;
```

```
printf("42 / 0 = %d, and the program is still running\n", q);
```

```
Linux, x86-64:
about to compute 42 / 0
Floating point exception (core dumped)
macOS, Apple Silicon:
about to compute 42 / 0
42 / 0 = 0, and the program is still running
```

On x86-64, `idivl` raises exception 0 and the kernel ends the program with the signal `SIGFPE`. The message says "floating point" for historical reasons. On arm64 the divide instruction raises no exception and returns 0. In C, dividing by zero is undefined behavior (Lecture 8), so neither answer is wrong.

Traps: System Calls Are Numbered

A **system call** is a request to the kernel, made with the `syscall` trap. The number in `%rax` says which service. Some Linux x86-64 numbers:

Number	Name	What it does	Lecture
0	`read`	read bytes from a file	13
1	`write`	write bytes to a file	today
9	`mmap`	map memory or a file	10
39	`getpid`	get this process's ID	today
57	`fork`	create a process	18
60	`exit`	end this process	today

macOS numbers its system calls differently.

Arguments Go in Registers

What	Function call (Lecture 7)	System call
which one	the `call` address	number in `%rax`
arguments 1 to 3	`%rdi`, `%rsi`, `%rdx`	`%rdi`, `%rsi`, `%rdx`
argument 4	`%rcx`	**`%r10`**
arguments 5, 6	`%r8`, `%r9`	`%r8`, `%r9`
result	`%rax`	`%rax`

Only argument 4 moves. The `syscall` instruction itself stores the return address in `%rcx` and the flags in `%r11`, so anything in `%rcx` is gone before the kernel sees it.

Hello, World With No C Library

This program uses no C library. It makes two system calls itself:

```
msg:    .ascii  "hello, world\n"
        len = . - msg               # 13 bytes

        .globl  _start
_start:                             # the kernel starts us here
        movq    $1, %rax            # system call 1 is write
        movq    $1, %rdi            # argument 1: fd 1, the screen
        leaq    msg(%rip), %rsi     # argument 2: address of the bytes
        movq    $len, %rdx          # argument 3: how many bytes
        syscall
```

```
        movq    $60, %rax             # system call 60 is exit
        movq    $0, %rdi              # argument 1: exit status 0
        syscall                       # does not return
```

There is no `main`. Normally the C library's startup code, `_start`, calls `main`. Without the library, the kernel starts the program at `_start`.

Run It: Two System Calls, Nothing Else

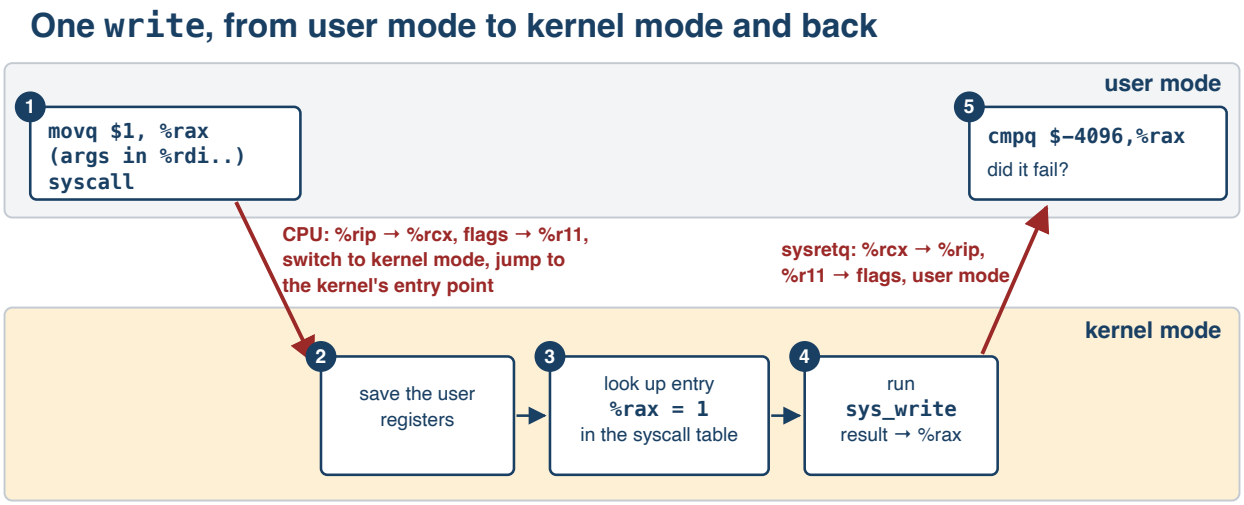
`strace` (Linux only) runs a program and prints each system call it makes:

```
gcc -nostdlib -static 04-hello.s -o 04-hello
strace ./04-hello
```

```
execve("./04-hello", ["./04-hello"], 0x7ffdaacb5da0 /* 17 vars */) = 0
write(1, "hello, world\n", 13)          = 13
exit(0)                                 = ?
```

`execve` is the shell starting the program. Then come exactly the two calls in the source. `write` returned 13, the number of bytes written.

What `syscall` Does, Step by Step



Red steps are done by the CPU itself. Everything in the kernel lane is ordinary code that the kernel wrote.

The **system call table** in step 3 is an array of function pointers inside the kernel, indexed by `%rax`.

How a System Call Reports an Error

When a system call fails, the kernel returns a negative error code in `%rax`, such as -2 for a file that does not exist. The C library's wrapper checks for any value from -4095 to -1 with one unsigned comparison (the `ja` from slide 1). If it finds one, it stores the positive code in the variable **`errno`** and returns -1 to your program:

```
int fd = open("/no/such/file", O_RDONLY);
if (fd == -1)
    perror("open");       // prints: open: No such file or directory
```

```
$ strace -e trace=openat ./05-errno
openat(AT_FDCWD, "/no/such/file", O_RDONLY) = -1 ENOENT (No such file or directory)
```

Code 2 is `ENOENT`. Read `errno` only after a failure, and right away.

Run It: 1000 `printf` Calls, How Many `write`s?

`07-buffering.c` is one loop:

```c
for (int i = 0; i < 1000; i++)
    printf("line %d\n", i);
```

```
strace -c -e trace=write ./07-buffering              # to the terminal
strace -c -e trace=write ./07-buffering > /dev/null  # to a file
```

Output goes to	`printf` calls	`write` calls
the terminal	1000	1000
a file	1000	3

`printf` collects text in a buffer and calls `write` only to empty it. For a terminal it empties the buffer at every newline. For a file it waits for 4096 bytes. The 1000 lines are 8890 bytes, so 3 writes.

Run It: What a System Call Costs

`08-syscall-cost.c` times one million calls each of a tiny function, a system call that does almost nothing, and a system call that writes one byte:

```c
__attribute__((noinline)) static int tiny(int x) { return x + 1; }
// each loop is timed with clock_gettime(CLOCK_MONOTONIC); fd is /dev/null
for (int i = 0; i < N; i++) sink = tiny(sink);              // function call
for (int i = 0; i < N; i++) sink = getppid();               // system call
for (int i = 0; i < N; i++) sink = (int)write(fd, "x", 1);  // system call
```

Machine	function call	`getppid()`	1-byte `write`
AMD Ryzen 7 8845HS (2024)	1.1 ns	241 ns	295 ns
Intel Core i5-6300HQ (2015)	1.5 ns	986 ns	1063 ns

A system call that does almost nothing takes as long as several hundred function calls. The CPU changes mode twice, and the kernel saves registers and checks every pointer it is given. So do more work per call. That is why `printf` buffers.

A Process Is a Running Program

A **program** is a file on disk. A **process** is one running instance of it. Three terminals running `vim` are one program and three processes.

The kernel keeps each process's **context**: its registers, its memory, its open files, and its **process ID** (PID).

The kernel also gives every process two illusions. The first is that the process has the whole CPU: its instructions run one after another as if no other program existed. In reality the kernel stops it and runs other processes in between. The rest of this lecture explains how. The second is that the process has all of memory: its addresses never collide with another program's. In reality every process shares one physical memory, and the kernel maps each process's addresses onto it. Lecture 10 explains that part, called virtual memory.

Run It: Every Process Has a Number

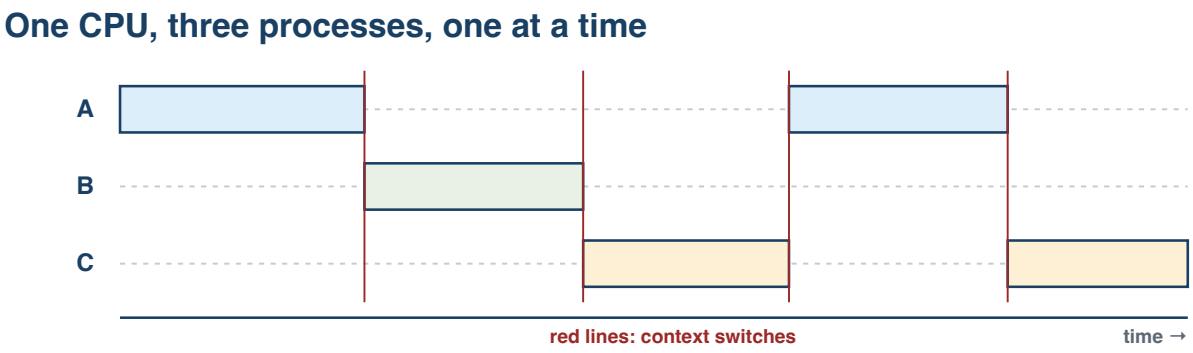
`09-pid.c` asks the kernel for two numbers:

```c
printf("my PID:        %d\n", (int)getpid());
printf("my parent PID: %d\n", (int)getppid());
```

```
$ ./09-pid                    $ ./09-pid
my PID:        3089851        my PID:        3089874
my parent PID: 3089850        my parent PID: 3089850
```

Each run is a new process with a new PID. Both have the same **parent**, the shell (check with `echo $$`). To run a command, the shell calls `fork` to copy itself, the copy calls `execve` to become the program, and the shell calls `wait` until it ends. Lecture 17 covers all three.

One CPU, Many Processes



Only one process runs at any moment. At each red line the kernel stops one process and starts another.

The kernel runs A for a while, then stops A and runs B, then C. When it stops a process, it saves the exact point the process had reached. When it runs that process again, the process continues from that point. So each process still runs its own instructions in their original order, and it cannot tell that it was stopped. That order of instructions is called the process's **logical control flow**.

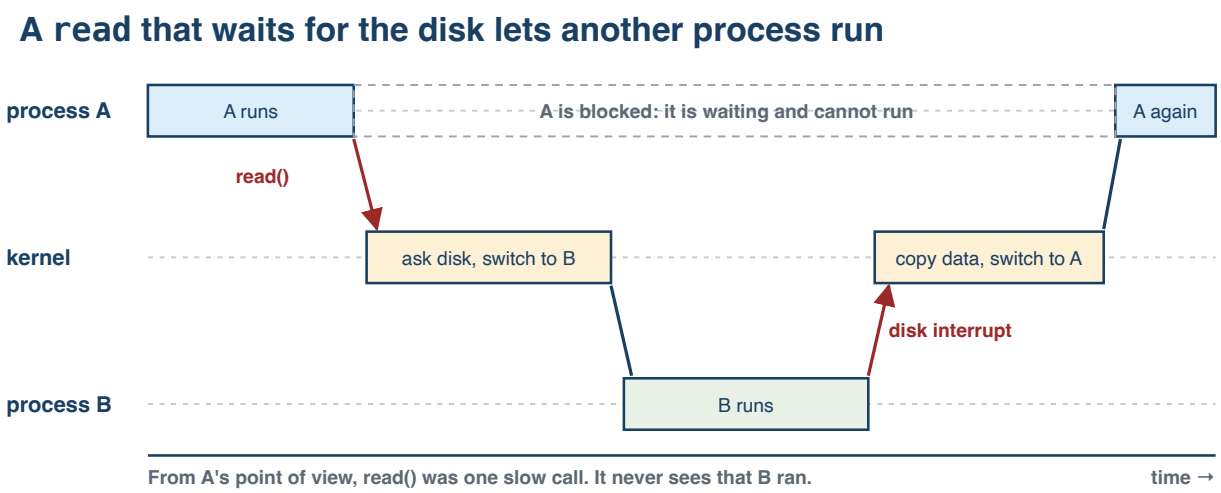
Two processes are **concurrent** when one starts before the other has finished, as A and C do here. Two processes run in **parallel** only when both are executing at the same moment, which needs two CPU cores. With one core, processes can be concurrent but never parallel.

The Context Switch

A **context switch** is the kernel stopping one process and starting another. To switch from A to B, the kernel saves A's registers, switches to B's memory mappings, loads B's saved registers, and returns to user mode in B.

A context switch happens inside the kernel, so it always starts with an exception. There are two usual causes. Either A made a system call that has to wait, for example for the disk, or a timer interrupt arrived and the kernel decided that A has had its turn.

A `read` That Has to Wait



A disk read can take milliseconds, time for millions of instructions. The kernel runs B instead of letting the CPU sit idle.

Run It: Counting Context Switches

The kernel counts two kinds of context switch for each process. A **voluntary** switch happens when the process asks the kernel to wait for something. An **involuntary** switch happens when the kernel takes the CPU away from a process that was still ready to run. `10-context-switches.c` reads both counts with `getrusage` before and after each of two loops:

```c
for (int k = 0; k < 100; k++)
    usleep(1000);                   // 1 ms: ask the kernel to wake us later

volatile unsigned long spin = 0;
time_t end = time(NULL) + 2;
while (time(NULL) < end)            // never asks the kernel to wait
    spin++;
```

What the Context Switch Counts Show

```
$ ./10-context-switches      # Linux
sleeping 100 times:   100 voluntary     0 involuntary
spinning 2 seconds:     0 voluntary    42 involuntary
```

Each call to `usleep` asks the kernel to wake the process 1 ms later, so the process gives up the CPU: 100 calls, 100 voluntary switches. The loop never asks the kernel for anything, but the timer interrupt still gave the kernel the CPU, and the kernel took it away from the loop 42 times. The count varies from run to run. When more copies run than the machine has cores, it reaches the hundreds.

AI Systems Connection

Is a system call always more expensive than a function call?

A *CUDA kernel* is a function that runs on the GPU, not the operating system kernel. The CPU starts one by calling a function in NVIDIA's library, a *launch*. A launch is not a system call: at startup the operating system maps a command buffer and one GPU register into the program's memory, and each launch is ordinary stores there. `11-launches.cu`, on the AMD laptop from the system call table:

What the program does	Time
one system call, `getppid()`	0.24 µs
launch one empty kernel	1.7 µs, about 7 system calls
1,000,000 additions as 1,000 small kernels	1,630 µs, about 1,000 launches × 1.7 µs
the same 1,000,000 additions as 1 big kernel	10 µs

So no: one launch, a plain function call, costs as much as 7 system calls. The small kernels take 160 times as long because the CPU spends the time launching while the GPU waits. Frameworks *fuse* small kernels into bigger ones: do more work per call, as with `printf`.

Think About It: `printf` on the GPU

A CUDA kernel may call `printf`. But printing needs a `write` system call, and the GPU cannot make system calls. How does this print anything? `12-gpu-printf.cu`, simplified:

```
__global__ void hello(void) {              // runs on the GPU, once per thread
    printf("GPU thread %d: hello\n", threadIdx.x);
}

int main(void) {                           // runs on the CPU
    hello<<<1, 4>>>();                     // launch 4 GPU threads
    printf("CPU: kernel launched\n");
    cudaDeviceSynchronize();               // the CPU waits for the GPU
    printf("CPU: done\n");
}
```

Predict before the next slide: in what order do the six lines appear? And what prints if the CPU never calls `cudaDeviceSynchronize()`?

`printf` on the GPU: The CPU Does the Printing

```
$ ./12-gpu-printf                          # on the RTX 4070 laptop
CPU: kernel launched
GPU thread 0: hello
GPU thread 1: hello
GPU thread 2: hello
GPU thread 3: hello
CPU: done
$ strace -f -e trace=write ./12-gpu-printf      # 18 writes; the one to stdout:
write(1, "GPU thread 0: hello\nGPU thread 1"..., 80) = 80    # made by the CPU
```

The GPU's `printf` only stores its text in a buffer in GPU memory, 1.7 MB on this machine. When the CPU waits for the GPU, NVIDIA's library copies the buffer out, and the CPU makes one `write` system call for all four lines.

- If the CPU never waits, only the two CPU lines print.
- With a million GPU threads, each printing one line, only 878,334 and 939,909 lines arrived in two runs. The rest were lost, and no error was reported.

Tip: part of the NVIDIA driver runs inside the operating system kernel, so reboot after a driver upgrade. Until then `nvidia-smi` reports a version mismatch.

The Lecture in One Slide

- Only **kernel mode** can use privileged instructions and devices. The only way in is an **exception**, which jumps to a handler the kernel chose.

- An **interrupt** comes from a device. A **trap** is a request the program makes on purpose. A **fault** is a failed instruction that the kernel may fix and run again. An **abort** is a hardware error that ends the program.
- A **system call** puts its number in `%rax` and runs `syscall`. It takes as long as hundreds of function calls. A failed call returns −1 and sets `errno`.
- A **process** is a running program. The kernel lets many processes share one CPU by switching between them.

Practice

1. Which exception class is each? (a) A key is pressed. (b) A program calls `write`. (c) A program first touches a heap page. (d) A program runs `hlt` in user mode.
2. For each case in question 1, does the handler return to the current instruction, return to the next one, or end the program?
3. Why does a system call's fourth argument go in `%r10` and not `%rcx`?
4. The kernel returns −13 from `openat`. What does `open` return, and what is in `errno`?

Practice II

5. Run `strace -e trace=getpid,getppid,write ./09-pid`, once to the terminal and once with `> out.txt`. How many `write` calls each time, and why?
6. `printf("x")`, with no newline, runs 10,000 times with output to the terminal. How many `write` calls do you expect? Check with `strace`.
7. A process on a one-core machine runs `while (1) { }`. How do other programs still get to run?
8. Run `strace ./06-printf`. It makes 35 system calls to print one line. Which one prints it, and what are most of the others doing?

Answers to Practice

1. (a) interrupt, (b) trap (a system call), (c) fault (a page fault), (d) fault (the CPU refuses an instruction only kernel mode may run).
2. (a) The next instruction: the program carries on as if nothing happened. (b) The next instruction, with the result in `%rax`. (c) The same instruction again, which now succeeds. (d) The program is ended with a segmentation fault, because the kernel cannot fix the problem.
3. The `syscall` instruction itself overwrites `%rcx`: it saves the return address there, and the flags in `%r11`. A fourth argument in `%rcx` would be gone before the kernel could read it.
4. `open` returns −1 and sets `errno` to 13, which is `EACCES`, "Permission denied". The C library turns the kernel's negative result into −1 plus `errno`.

Answers to Practice II

5. To the terminal: `getpid`, `write`, `getppid`, `write`, so 2 writes, because output to a terminal is line buffered and each `printf` ends with a newline. To a file: `getpid`, `getppid`, then one 46-byte `write` when the program exits, because output to a file is fully buffered.
6. 10 writes. With no newline, a terminal's buffer empties only when it is full. On Linux that buffer is 1024 bytes for a terminal, so 10,000 bytes make 9 full buffers plus one last flush when the program exits.
7. The timer interrupt. Every few milliseconds it hands the CPU to the kernel, and the scheduler can switch to another process: an involuntary context switch.
8. `write(1, "hello\n", 6)` prints the line. Most of the others are the dynamic loader opening `libc.so.6` and mapping it into memory (`openat`, `mmap`, `mprotect`, `close`), plus `malloc` setting up the heap (`brk`) and `fstat` on stdout to choose a buffer. The count depends a little on where the output goes: 35 to a pipe or a terminal, 36 to `/dev/null`.

Reading

- CS:APP §8.1: Exceptions

- CS:APP §8.2: Processes
- CS:APP §8.3: System Call Error Handling
- `man 2 syscall` and `man 2 intro`